sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the ServiceRegistry system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation echo . . . 4
3.2 operation power . . . 4
3.3 operation status . . . 4

4 Data Models **6**
4.1 struct Observation . . . 6
4.2 Primitives . . . 6

5 Go and Rust implementation variants **7**
5.1 Go implementation . . . 7
5.2 Rust implementation . . . 7

6 References **7**

7 Revision History **8**
7.1 Amendments . . . 8
7.2 Quality Assurance . . . 8

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the AMQP binding of the native `sysMonitor` service of ServiceRegistry.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

2.1 Interface registration

With `sysmonitor.enabled=true`, ServiceRegistry registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
```

```
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.ServiceRegistry.ServiceRegistry",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

3 Service Operations

The publisher connects to the broker (`sysmonitor.broker.address`, default `amqp-broker`; port `sysmonitor.broker.port`, default 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. The local cloud name is taken from `sysmonitor.local.cloud` or the system certificate. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

3.1 PUBLISH `<Cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

3.2 PUBLISH `<Cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

3.3 PUBLISH `<Cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "ServiceRegistry-echo-1792670400000-42",
  "providerSystem": "ServiceRegistry",
  "serviceInstance": "ServiceRegistry",
```

```
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "ServiceRegistry is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
}
```

Listing 2: An echo observation.

4 Data Models

4.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power` or `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

4.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Default 8443 (server.port in application.properties). Override with server.port, SERVER_PORT=N or -server.port N; the host is server.address. The native registry API, outsourced authentication, JSON persistence, optional AMQP telemetry and differences from Java are specified in `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry/UPSTREAM-README.md`. It has no MQTT application API according to that profile. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/serviceregistry`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ServiceRegistry`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-sysMonitor-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	